

`torch.jit.annotations.parse_type_line` is not safe (command injection) even it seems already deprecated. #88868

Closed

Lyutoon opened on Nov 11, 2022 · edited by pytorch-bot

Edits · ···

Describe the bug

In `torch.jit.annotations`, it looks like there are some functions that are deprecated, but still retain code, which may lead to some backdoors, especially since some of these functions still use `eval` while implementing.

But now I'm not sure if there are some features (jit decorator) in some version of pytorch are still using this function `parse_type_line` or `get_signature` which calls `parse_type_line`, if so, it can cause RCE, if not, maybe someone can also leave a backdoor by calling this function while writing code and share it to the people.

```
import torch
```

```
torch.jit.annotations.parse_type_line('# type: __import__("os").system("ls") -> 234',  
None, 1)
```

Versions

master

cc [@ezyang](#) [@gchanan](#) [@zou3519](#) [@EikanWang](#) [@jgong5](#) [@wenzhe-nrv](#) [@sanchitintel](#)



malfet added

triage review

topic: security

on Nov 12, 2022

malfet on Nov 15, 2022 · edited by malfet

Edits · Contributor ···

Marking for triage review(and not assigning `oncall: jit` yet otherwise it will disappear to the void) to discuss what to do with those kinds of security issues, which, in my opinion, is pretty minor: if one have an access to local Python runtime they can do anything they want.

Lyutoon on Nov 15, 2022 · edited by Lyutoon

Edits · Author ···

Marking for triage review(and not assigning `oncall: jit` yet otherwise it will disappear to the void) to discuss what to do with those kinds of security issues, which is in my opinion is pretty minor: if one have an access to local Python runtime they can do anything they want.

Yes, this seems not a very urgent bug, but we still need to pay attention to these dangerous functions such as `eval`. And in `CVE-2022-0845`, this bug is also caused by using `eval` to parse the args so it leads to code injection, and it seems also must have an access to local python. To be honest, I don't know how people think about these kind of problems, but we need to pay more attention :p. So we need a discuss about it whether it can be considered as a big security problem.

 **malfet** added `high priority` on Nov 15, 2022

 **malfet** self-assigned this on Nov 15, 2022

 malfet on Nov 15, 2022

Contributor ...

We should have a doc marking unsafe function and safe versions of the same.
And also, probably should not use eval, if possible.

 **albanD** added `oncall: jit` and removed `triage review` on Nov 15, 2022

 **malfet** added `triage review` and removed `triage review` on Nov 15, 2022

 Lyutoon on Nov 15, 2022

Author ...

We should have a doc marking unsafe function and safe versions of the same. And also, probably should not use eval, if possible.

That's right, there was also a cve
(<https://github.com/tensorflow/tensorflow/blob/master/tensorflow/security/advisory/tfsa-2022-060.md>) in tensorflow that used `eval` in `saved_model_cli` and caused code injection. Also I've found that PaddlePaddle has also `eval` problems
(<https://github.com/PaddlePaddle/Paddle/blob/develop/security/advisory/pdsa-2022-002.md>). But sometimes, if we do not use `eval`, the code will become much more complex. Maybe developers can just add a critical check about the parameters of the function to avoid this problem easily (if possible).

 **malfet** added `triaged` on Nov 17, 2022

 malfet on Nov 17, 2022

Contributor ...

It seems reasonable, that valid type annotations should not have any function/method calls, so splitting it into `compile` (which is safe)->error on function calls-> `eval` would secure the deprecated codebase without overcomplicating deprecated code too much

1



malfet mentioned this [on Nov 17, 2022](#)

[\[JIT\]\[Security\] Do not blindly eval input string #89189](#)

13 remaining items

Load more

ezyang on Dec 6, 2022

Contributor ...

Definite not via user controlled tensor inputs. There is no vector for type annotations there

malfet on Dec 6, 2022

Contributor ...

I guess the problem here is that `torch.jit.annotations.parse_type_line` looks like a public API, which means that dev can use it in their code to extract function annotation and arbitrary code execution feels like a very unexpected side-effect of calling such function.

abustamante-coveo on Dec 6, 2022

...

hello, is there an ETA on 1.13.1?

malfet on Dec 7, 2022

Contributor ...

[@abustamante-coveo](#) tentatively Dec 15th, see [#89855](#)

1



kulinseth added a commit that references this issue [on Dec 10, 2022](#)

[\[JIT\]\[Security\] Do not blindly eval input string \(pytorch#89189\)](#) ...

e9b5d8f



dhrubo-os mentioned this [on Dec 13, 2022](#)

✓ [CVE-2022-45907 \(High\) detected in torch-1.12.1-cp37-cp37m-manylinux1_x86_64.whl, torch-1.13.0-cp37-cp37m-manylinux1_x86_64.whl - autoclosed opensearch-project/opensearch-py-ml#49](#)

abustamante-coveo on Dec 16, 2022

...

hello, Snyk still marks torch as having a critical vulnerability even after the 1.13.1 patch. Was this vulnerability supposed to be patched on that version?

3

vikcher on Dec 17, 2022

hello, Snyk still marks torch as having a critical vulnerability even after the 1.13.1 patch. Was this vulnerability supposed to be patched on that version?

Looks like their vulnerability database is not updated after the fix. It shows that the fix is pushed but not published:

<https://security.snyk.io/vuln/SNYK-PYTHON-TORCH-3149871>



 **gradientsky** mentioned this [on Jan 6, 2023](#)

  [\[WIP\]\[DO NOT MERGE\] Update version of torch, lightning, transformers in multimodal autogluon/autogluon#2479](#)

 [Update torch to 1.13.1+ autogluon/autogluon#2627](#)



 **i-chvets** mentioned this [on Jan 26, 2023](#)

 [Update images to address CVEs 24.01.23 canonical/notebook-operators#109](#)



 **ganler** mentioned this [on Feb 9, 2023](#)

 [\[Crash\] torch.searchsorted with out-of-bound sorter #91606](#)



 **saitcakmak** mentioned this [on Feb 15, 2023](#)

 [Raise torch lower version bound to >=1.13.1 \(CVE-2022-45907\) meta-pytorch/botorch#1673](#)



 **jongwook** mentioned this [on Mar 7, 2023](#)

 [Pytorch 1.7 contains vulnerability that is fixed in upgraded version openai/CLIP#333](#)



 **paul-gerste-sonarsource** mentioned this [on Apr 15, 2025](#)

 [Code Injection via torch.jit.annotations.parse_type_line\(\) #151233](#)

Sign up for free to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Assignees



malfet

Labels

high priority

oncall: jit

topic: security

triaged

Type

No type

Projects

No projects

Milestone

✔ 1.13.1

Closed on Dec 20, 2022, 100% complete

Relationships

None yet

Development

 Code with agent mode

▼

🔗 [JIT][Security] Do not blindly eval input string
pytorch/pytorch

Participants



+5